

Motion Planning Lecture 8

Kinodynamic Planning: SST*, AO-x

Geometric Planning: Informed RRT*, BIT*

Wolfgang Hönig (TU Berlin) and Andreas Orthey (Realtime Robotics)

June 4, 2025

Recap

- Tree-based motion planning: RRT (probabilistic complete (PC), but suboptimal)
- Asymptotic Optimality (AO)
- RRT* introduces rewiring (probabilistic complete and asymptotically optimal)
- Proof sketch
 - PC RRT (by induction; series of connectable balls)
- kinodynamic RRT

Revisiting RRT*

Recap RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [1]

Recap RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [1]
- Parent of $\mathbf{q}_{\text{new}}$: May use other parent than $\mathbf{q}_{\text{near}}$ with lowest cost (within neighborhood of $\mathbf{q}_{\text{new}}$)

Recap RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [1]
- Parent of $\mathbf{q}_{\text{new}}$: May use other parent than $\mathbf{q}_{\text{near}}$ with lowest cost (within neighborhood of $\mathbf{q}_{\text{new}}$)
- Rewire edges: Use $\mathbf{q}_{\text{new}}$ as a new parent, for neighboring configurations, if it reduces costs

Recap RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [1]
- Parent of $\mathbf{q}_{\text{new}}$: May use other parent than $\mathbf{q}_{\text{near}}$ with lowest cost (within neighborhood of $\mathbf{q}_{\text{new}}$)
- Rewire edges: Use $\mathbf{q}_{\text{new}}$ as a new parent, for neighboring configurations, if it reduces costs
- Neighborhood radius depends on tree size:

$$r(|\mathcal{V}|) = \gamma \left(\frac{\log |\mathcal{V}|}{|\mathcal{V}|} \right)^{\frac{1}{d+1}}$$

Recap: RRT PC Proof sketch

Summary

- Assumption A: There is a feasible path
- Assumption B: It has ϵ clearance
- Assumption C: Sampling is dense

Proof sketch

- Put δ -spaced balls onto feasible path (depending on ϵ)
- Execute induction proof
 - Proof that the first ball is reached (trivial)
 - Proof that you reach ball B_{k+1} from B_k (main part)

Question

What if we replace "feasible path" with "optimal path". Does the proof still hold?

Note

- There is no guarantee that you make a connection from B_k to B_{k+1} (there might be a different nearest neighbor)

This is why this is not an optimality proof!

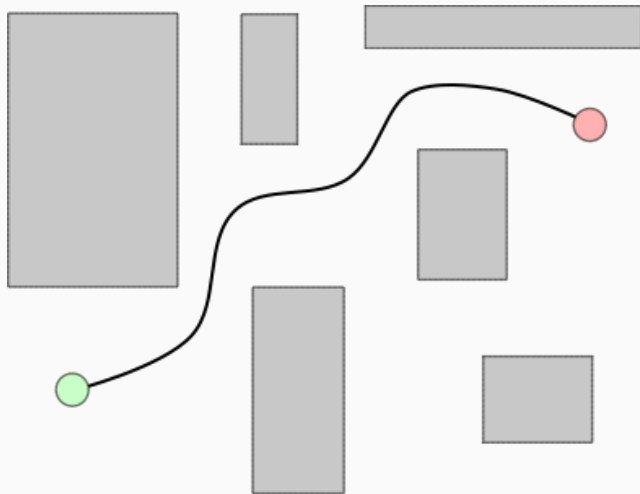
Question

How do we fix this proof for optimality?

Revisiting RRT*

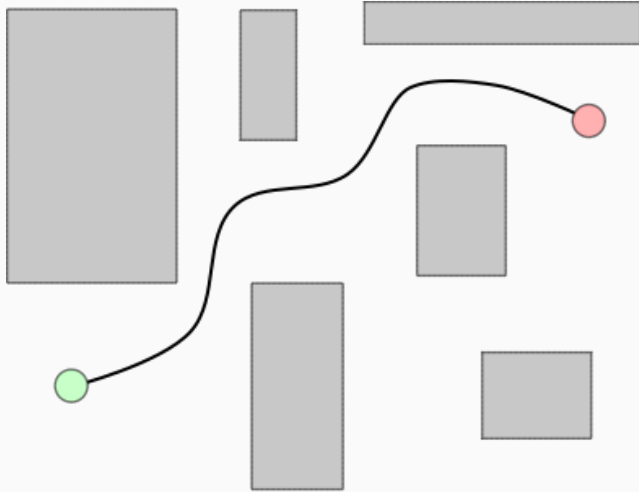
Asymptotically optimal proof RRT*

Proof sketch



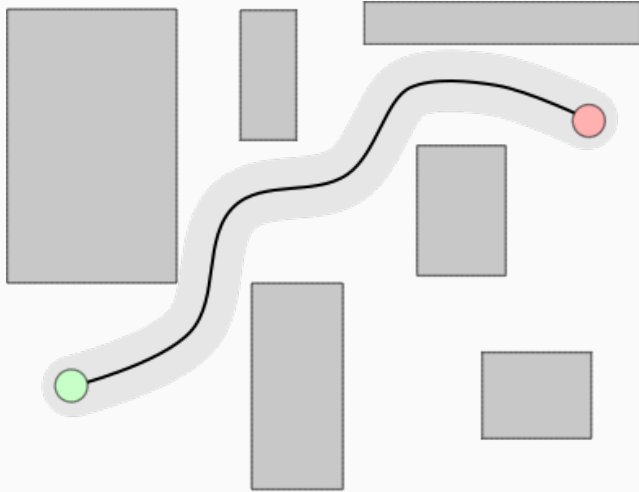
S Karaman and E Frazzoli, "Sampling-based Algorithms for Optimal Motion Planning", 2011 [2]

Proof sketch



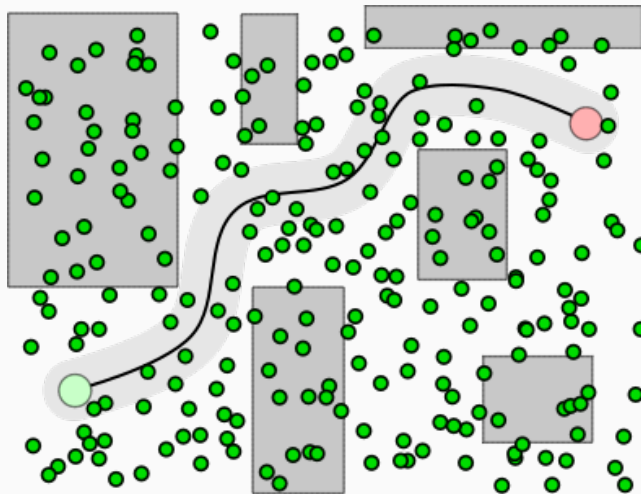
Assumption A: There exists an *optimal* path.

Proof sketch



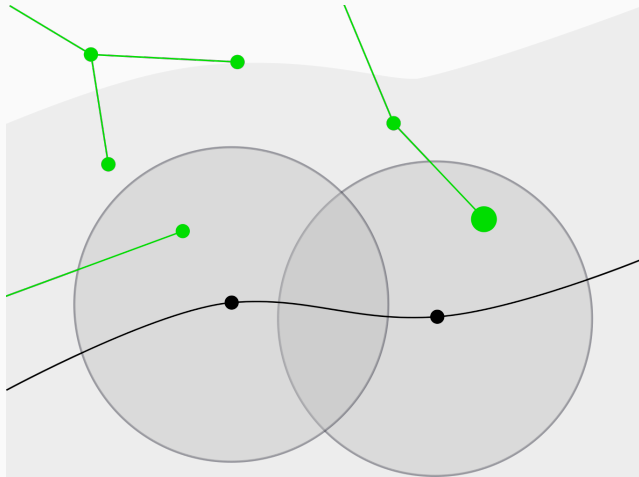
Assumption B: Optimal path has ϵ clearance.

Proof sketch



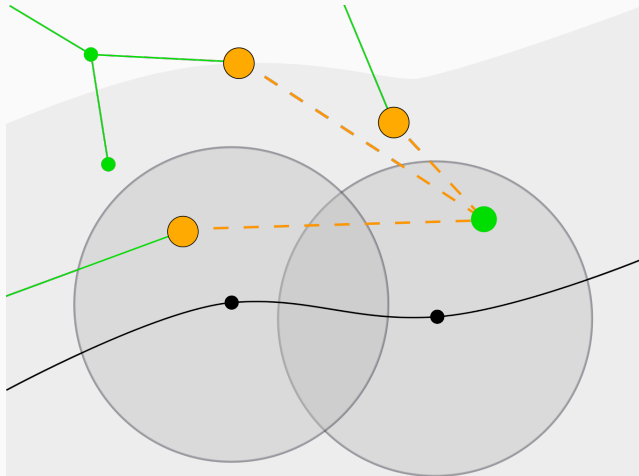
Assumption C: Sampling is dense.

Proof sketch

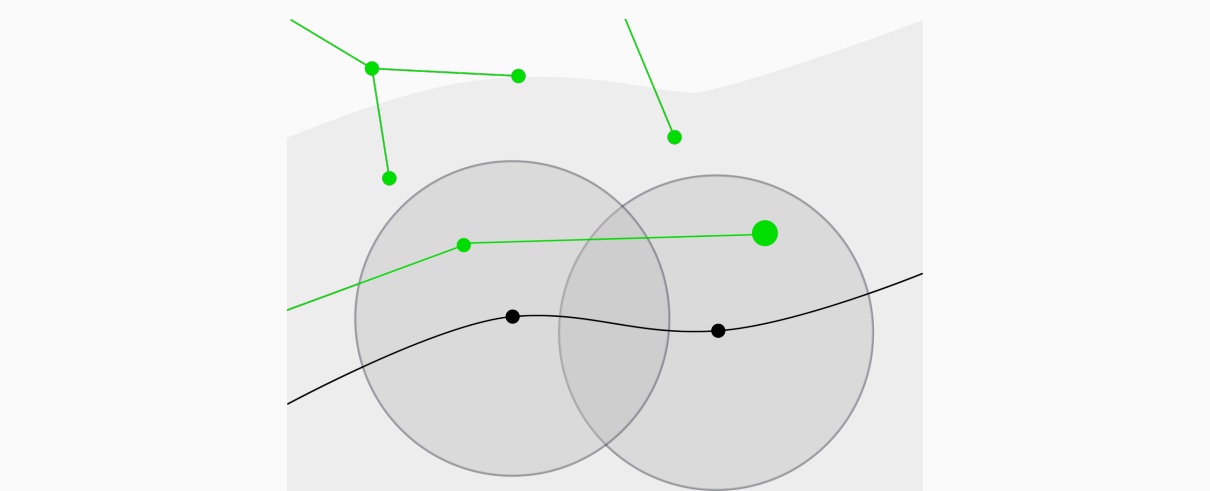


RRT might find wrong wiring.

Proof sketch



RRT* computes cost to come.

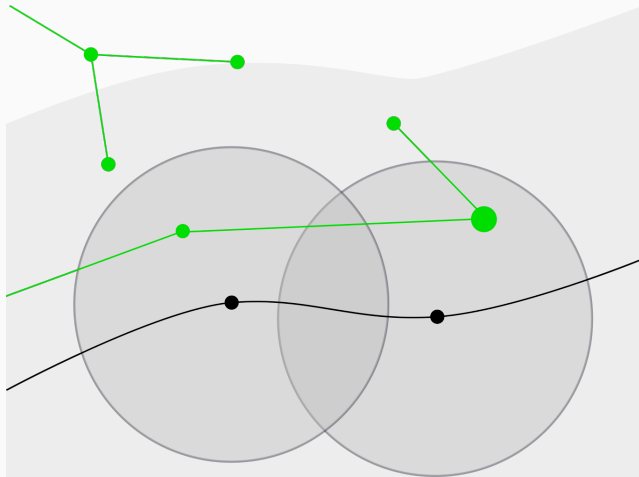


RRT* rewires accordingly.

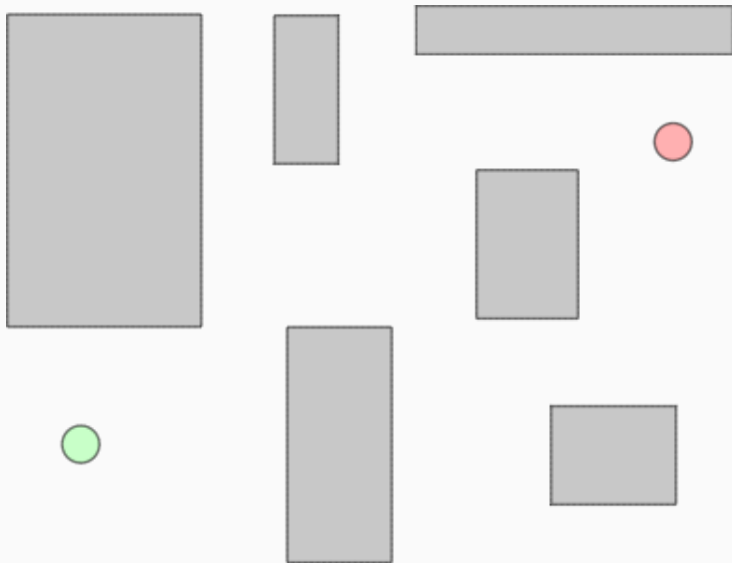
Proof idea

Use rewiring operation to show that we reach B_{k+1} always from B_k .

Proof sketch



Question: Do we need the second rewiring step?



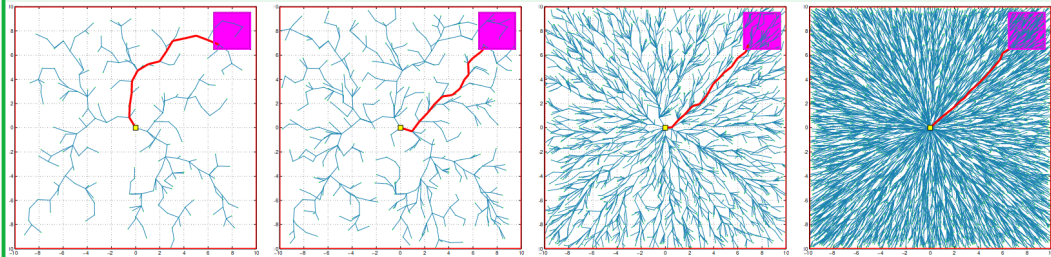
Two problems with RRT*

- Does not prioritize paths as A* does
- Once path is found, it still samples region which cannot improve solution

Informed optimal planning

Two problems with RRT*

RRT* example



Informed sampling

- Informed sampling restricts sampling to region which can improve solution
- Based upon concept of omniscient set

Reminder (see Lecture 3)

- Optimal cost-to-come $g(x)$ (minimal cost from start to x)
- Optimal cost-to-go $h(x)$ (minimal cost from x to goal)
- Optimal f-value $f(x) = g(x) + h(x)$ (minimal cost, constrained to go through x)

Definition omniscient set

Let c be the cost of a our current solution. Definition omniscient set:

$$X = \{x \in \mathcal{Q} \mid f(x) < c\}$$

Question

What does the omniscient set represent?

Definition informed set

Let c be the cost of a our current solution. Definition admissible informed set:

$$\hat{X} = \{x \in \mathcal{Q} \mid \hat{f}(x) < c\}$$

whereby $\hat{f} = \hat{g}(x) + \hat{h}(x)$ with $\hat{h}(x)$ being an admissible heuristic.

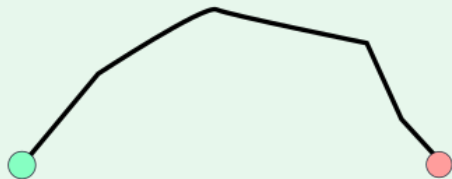
Definition L2-informed set

Let c be the cost of a our current solution. Definition admissible informed set:

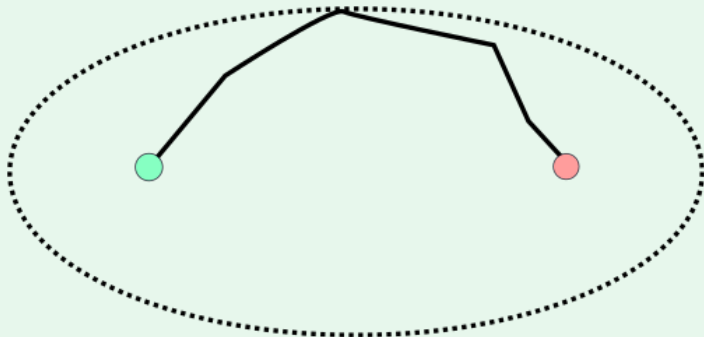
$$\hat{X} = \{x \in \mathcal{Q} \mid d(x_{start}, x) + d(x, x_{goal}) < c\}$$

For the L2-metric, this is called a **prolate hyperspheroid**

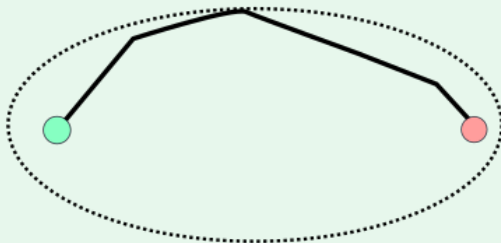
Informed Set



Informed Set



Informed Set



Informed Set



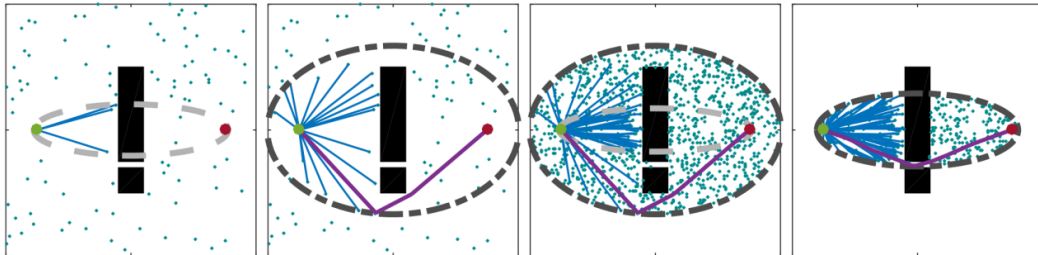
- Informed RRT* uses informed sets to sample more efficiently
- BIT* prioritizes node expansions based on cost similar to A*.
- In the Euclidean case with L2-norm, this becomes a growing hyperellipsoid.

JD Gammell et al., "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic", (2014)

JD Gammell et al. "Batch Informed Trees (BIT*): Informed asymptotically optimal anytime search", (2020)

Batch Informed Trees (BIT*)

BIT* example



Optimal Planning for Kinodynamic Systems

Recap: Meta Algorithm (for kinodynamic planning with forward propagation)

Meta($\mathbf{q}_{start}$, $\mathcal{Q}_{goal}$, $\mathcal{Q}$, $\mathcal{U}$, $\mathbf{f}$, t_{prop})

- $T = \text{InitializeTree}(\mathbf{q}_{start})$
- While Not Terminated
 - $\mathbf{q}_{select} = \text{SelectNode}(T, \mathcal{Q})$
 - $\mathbf{q}_{new} = \text{Propagate}(\mathbf{q}_{select}, \mathcal{U}, \mathbf{f}, t_{prop})$
 - $\text{MaybeAddConnection}(\mathbf{q}_{select}, \mathbf{q}_{new})$

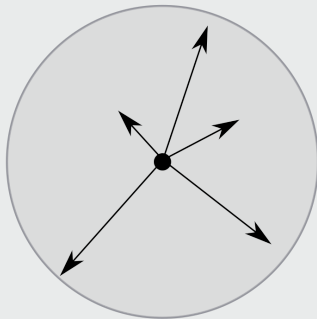
Kinodynamic RRT

- Select Node: Exploration First Selection
- Propagate Node: Guided Monte Carlo Propagation
- Maybe add connection: Collision-Free Checking

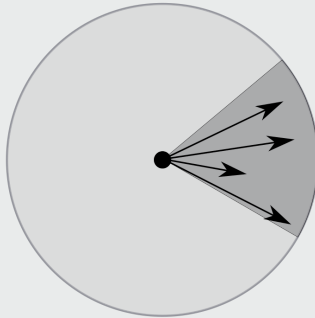
Optimal Planning for Kinodynamic Systems

kinodynamic RRT: PC Proof Sketch

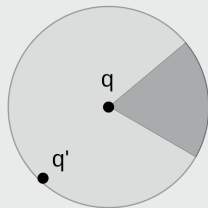
Small-space local controllability (SSLC) property



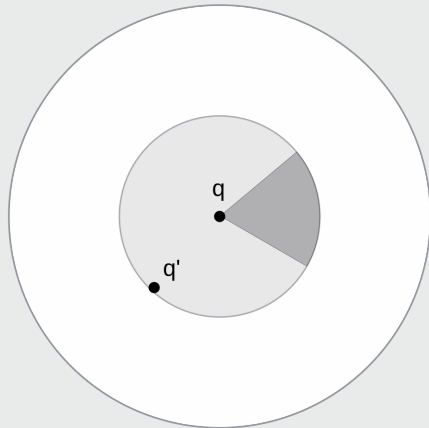
Small-space local controllability property



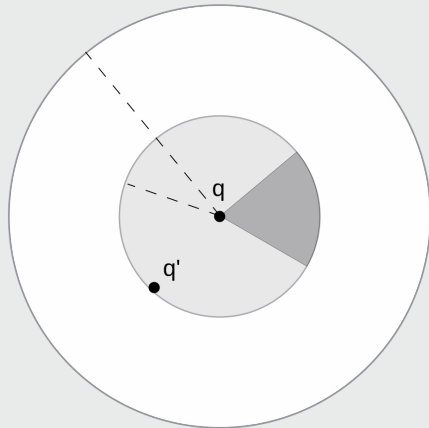
Small-space local controllability property



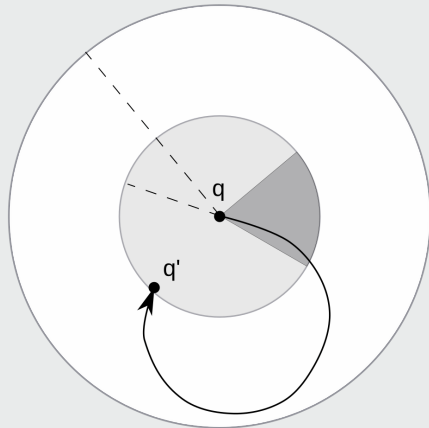
Small-space local controllability property



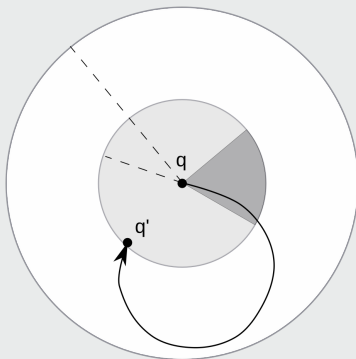
Small-space local controllability property



Small-space local controllability property

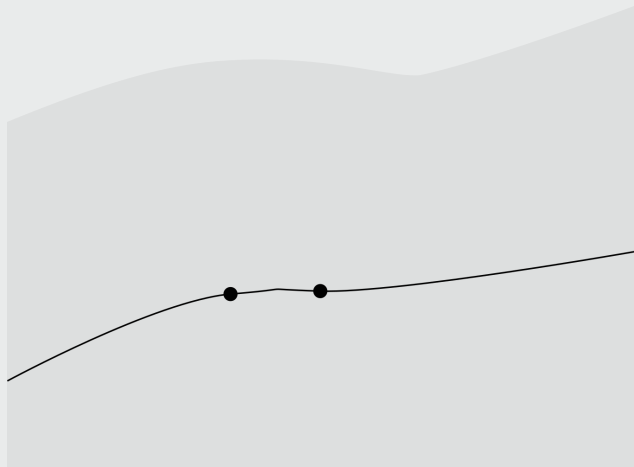


Small-space local controllability property

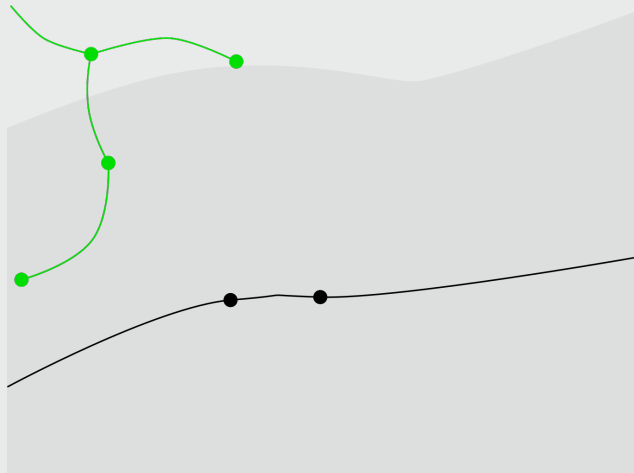


Small-space local controllability property: Any configuration q' at a distance less than δ is reachable from q by an admissible trajectory included in a ball of size $\epsilon > \delta$.

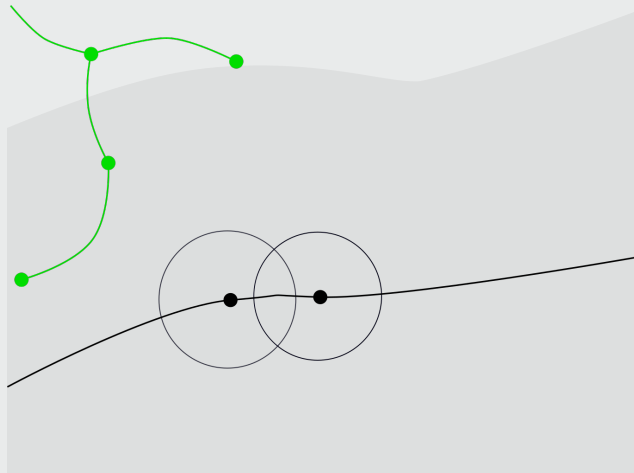
Series-of-balls argument in kinodynamic planning



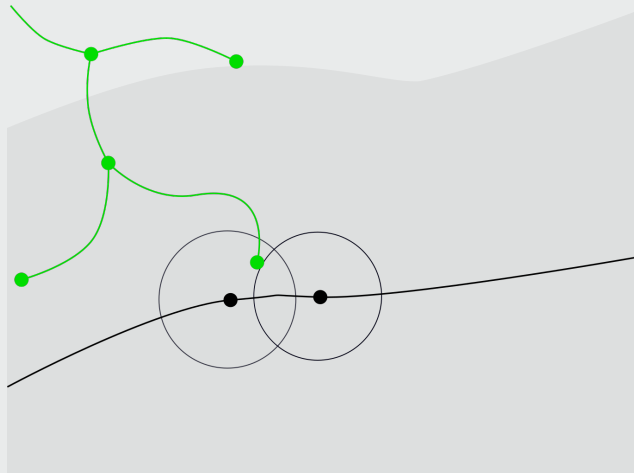
Series-of-balls argument in kinodynamic planning



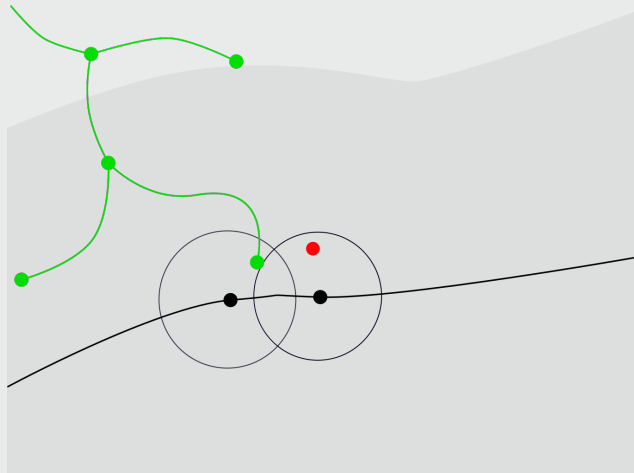
Series-of-balls argument in kinodynamic planning



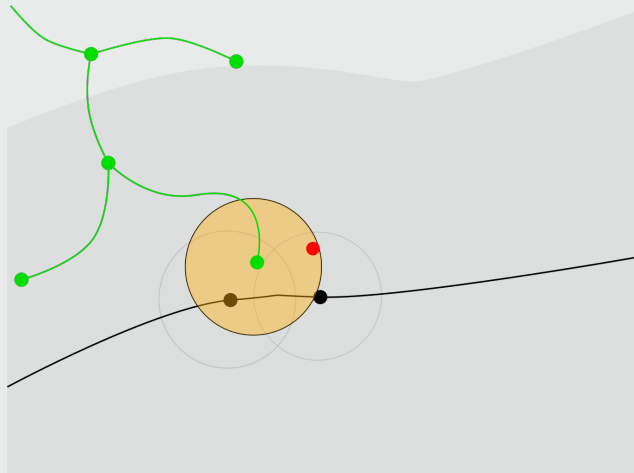
Series-of-balls argument in kinodynamic planning



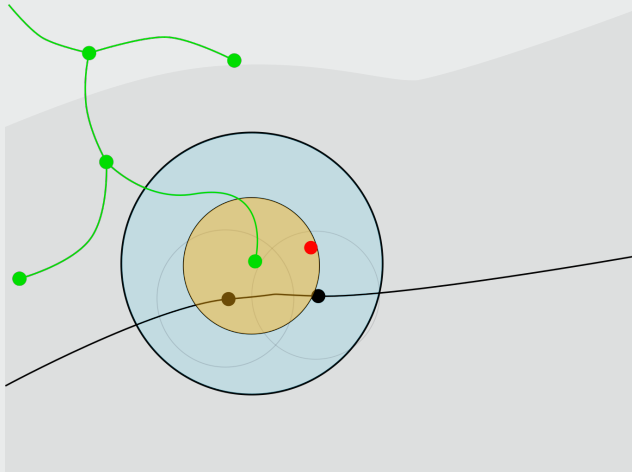
Series-of-balls argument in kinodynamic planning



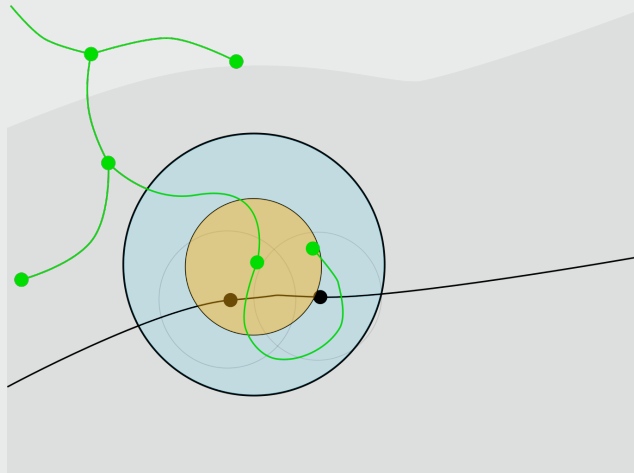
Series-of-balls argument in kinodynamic planning



Series-of-balls argument in kinodynamic planning



Series-of-balls argument in kinodynamic planning



Question

Is kinodynamic RRT also asymptotically optimal?

Optimal Planning for Kinodynamic Systems

Naive Random Trees

Naive Random Trees

Naive Random Trees

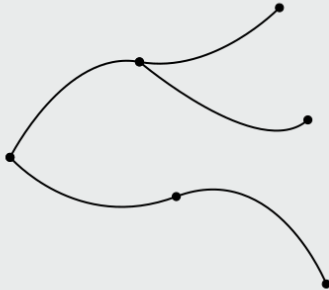
- Select Node:
Uniform Selection
- Propagate Node:
Monte Carlo
- Maybe add connection:
Collision-Free Checking

Algorithm 2: NAIVE_RANDOM_TREE($\mathbb{X}_f, \mathbb{U}, x_0, T_{prop}, N$)

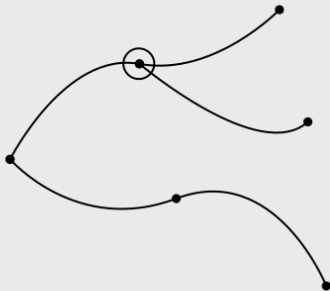
```
1  $G = \{\mathbb{V} \leftarrow \{x_0\}, \mathbb{E} \leftarrow \emptyset\};$   
2 for  $N$  iterations do  
3    $x_{selected} \leftarrow \text{Uniform\_Sampling}(\mathbb{V});$   
4    $x_{new} \leftarrow \text{MonteCarlo-Prop}(x_{selected}, \mathbb{U}, T_{prop});$   
5   if  $\text{CollisionFree}(\overline{x_{selected} \rightarrow x_{new}})$  then  
6      $\mathbb{V} \leftarrow \mathbb{V} \cup \{x_{new}\};$   
7      $\mathbb{E} \leftarrow \mathbb{E} \cup \{\overline{x_{selected} \rightarrow x_{new}}\};$   
8 return  $G(\mathbb{V}, \mathbb{E});$ 
```

Source: [3]

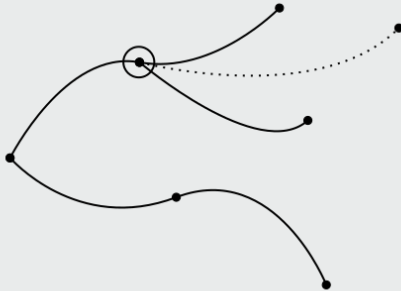
Select Node



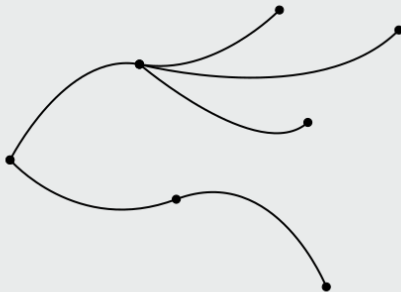
Select Node



Propagate Node



Propagate Node



Properties

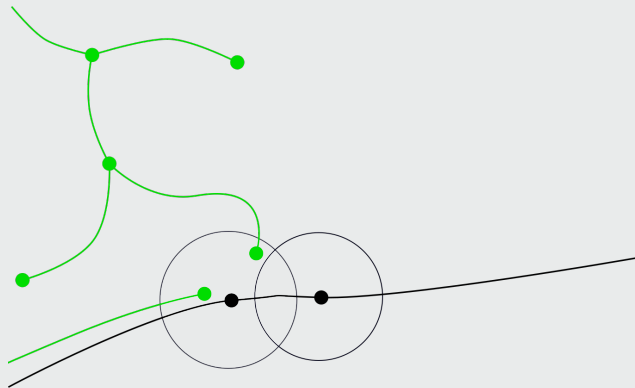
Naive Random Trees is asymptotically optimal

Question

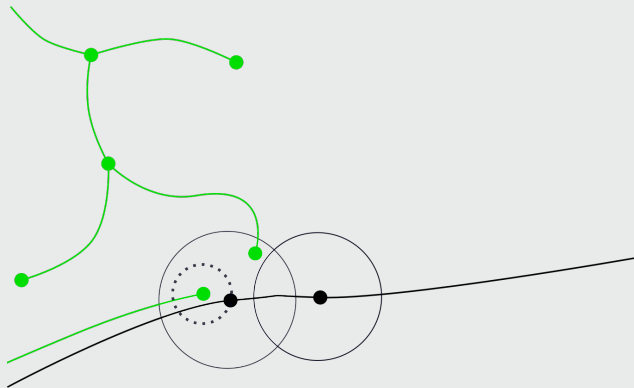
Why is that so?

Y Li, Z Littlefield, KE Bekris, "Asymptotically Optimal Sampling-based Kinodynamic Planning", 2016

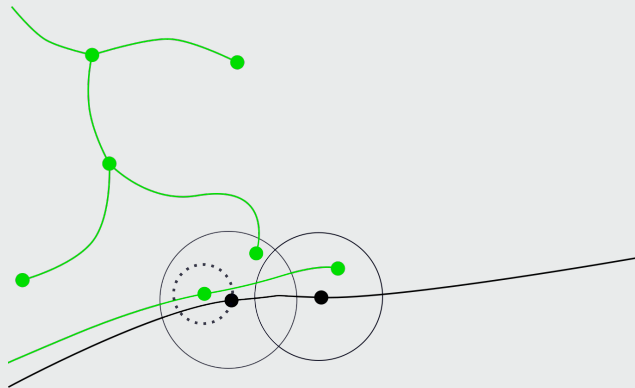
Proof sketch



Proof sketch



Proof sketch



Drawbacks

- Selection of nodes uninformative
- High memory footprint

Optimal Planning for Kinodynamic Systems

Stable sparse trees (SST*)

Sparse Stable Trees (SST)

Sparse stable trees (SST*)

- Select Node: Best First Selection
- Propagate Node: Monte Carlo
- Maybe add connection: Collision-Free Checking + Pruning

SST = Naive random trees with better selection and pruning

Sparse Stable Trees (SST): Best First Selection

- Select node with lowest cost-to-come within a neighborhood

Algorithm 6: Best_First_Selection
($\mathbb{X}, \mathbb{V}, \delta_{BN}$)

```
1  $x_{rand} \leftarrow \text{Sample\_State}(\mathbb{X});$   
2  $X_{near} \leftarrow \text{Near}(\mathbb{V}, x_{rand}, \delta_{BN});$   
3 If  $X_{near} = \emptyset$  return  $\text{Nearest}(\mathbb{V}, x_{rand});$   
4 Else return  $\arg \min_{x \in X_{near}} \text{cost}(x);$ 
```

Source: [3]

Sparse Stable Trees (SST)

SST Pruning

Pruning based on Witness set

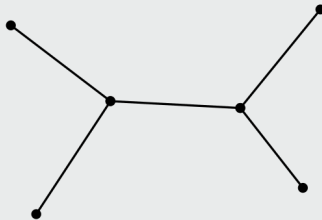
Witness Set

Set of states ($\mathcal{S} \subset \mathcal{Q}$) used as helper data structure.

Invariant for each $s \in \mathcal{S}$: only a single node of the search tree within radius $\delta_{\mathcal{S}}$ represents that state s and has best path cost from root.

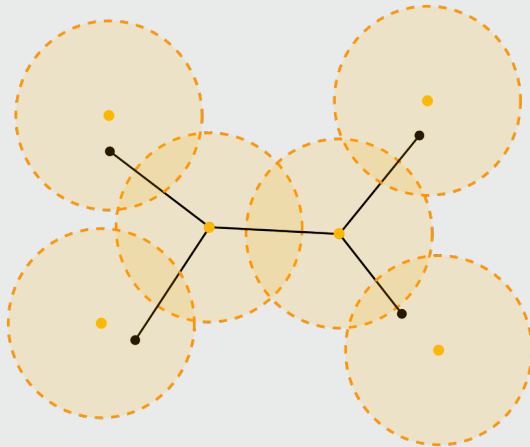
Sparse Stable Trees (SST)

Witness set: Search tree



Sparse Stable Trees (SST)

Witness set: witnesses $s \in \mathcal{S}$ in yellow



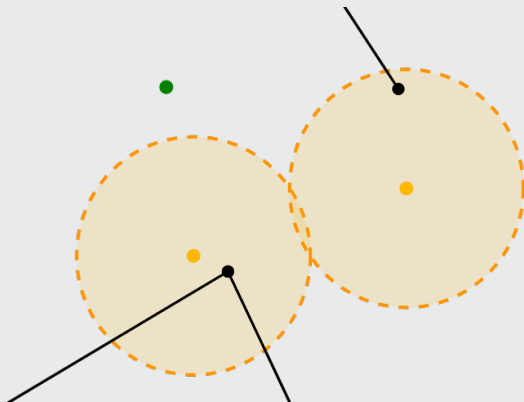
Sparse Stable Trees (SST)

Witness set

Adding connections

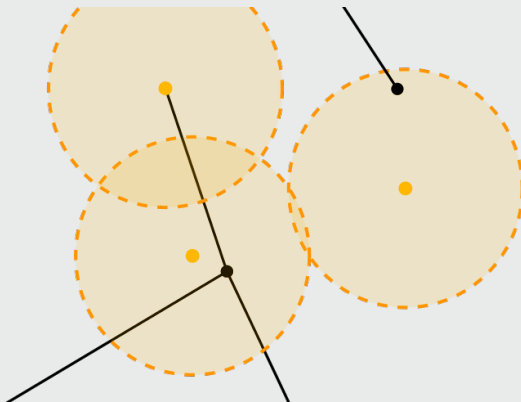
Sparse Stable Trees (SST)

Adding connection: Case 1 - No witness



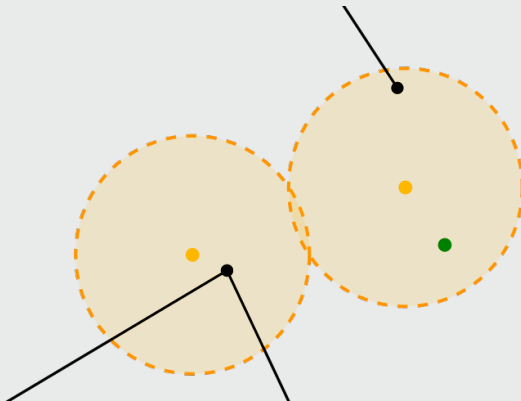
Sparse Stable Trees (SST)

Adding connection: Case 1 - No witness



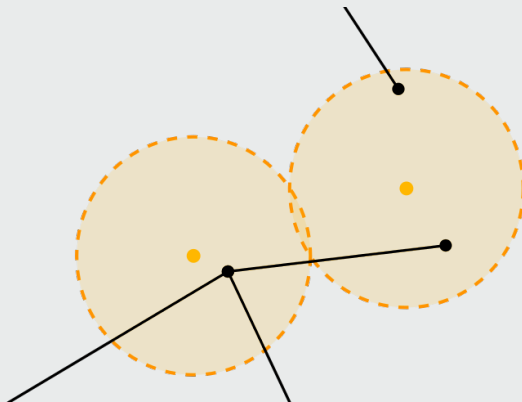
Sparse Stable Trees (SST)

Adding connection: Case 2 - Existing witness



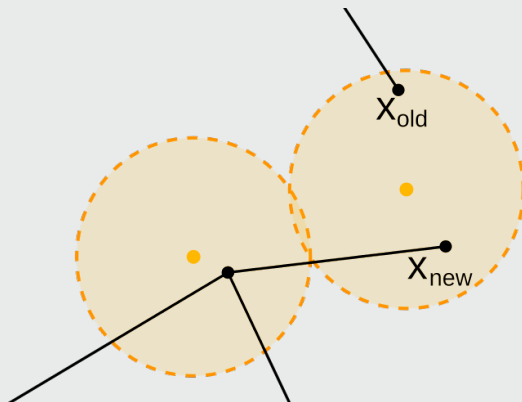
Sparse Stable Trees (SST)

Adding connection: Case 2 - Existing witness



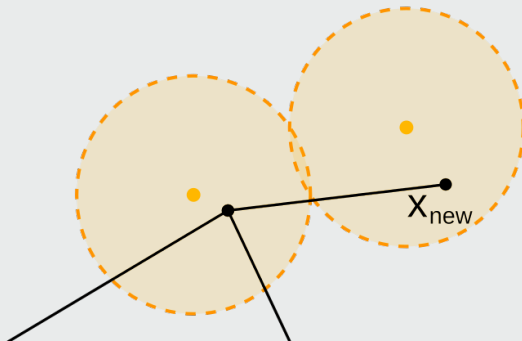
Sparse Stable Trees (SST)

Adding connection: Case 2 - Existing witness



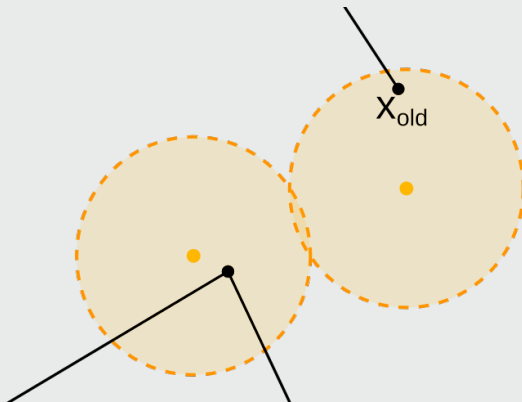
Sparse Stable Trees (SST)

Adding connection: Case 2a - Better cost



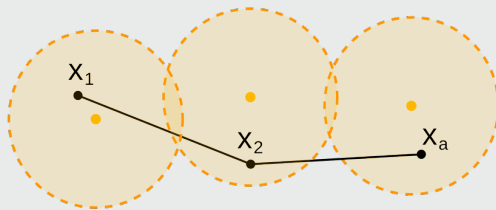
Sparse Stable Trees (SST)

Adding connection: Case 2b - Worse cost



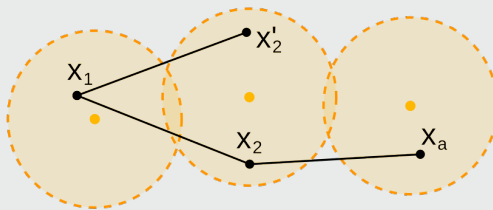
Sparse Stable Trees (SST)

Subtree pruning



Sparse Stable Trees (SST)

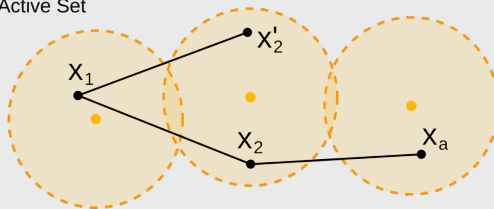
Subtree pruning



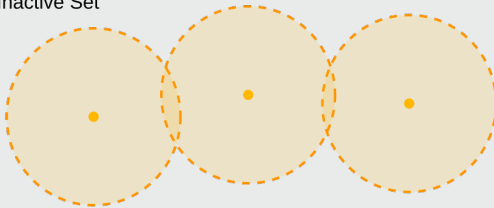
Sparse Stable Trees (SST)

Subtree pruning

Active Set

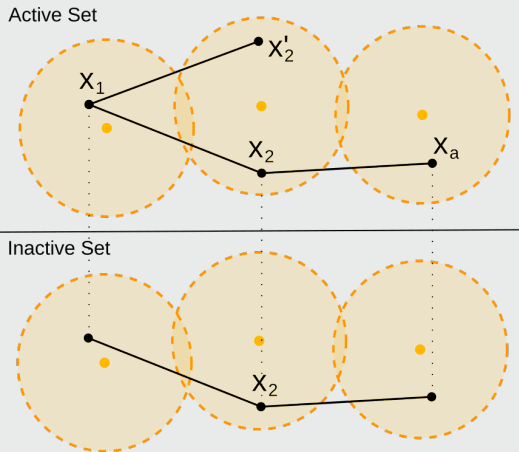


Inactive Set



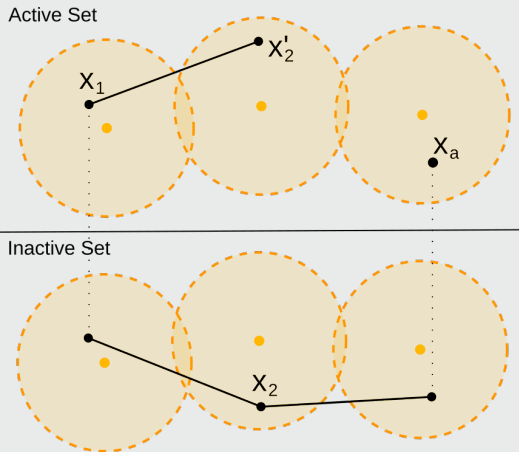
Sparse Stable Trees (SST)

Subtree pruning



Sparse Stable Trees (SST)

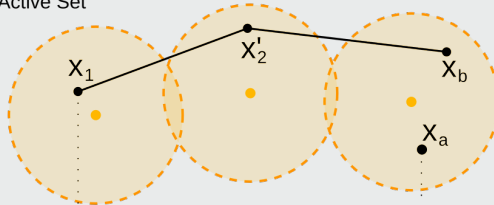
Subtree pruning



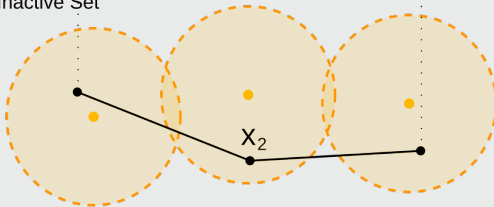
Sparse Stable Trees (SST)

Subtree pruning

Active Set



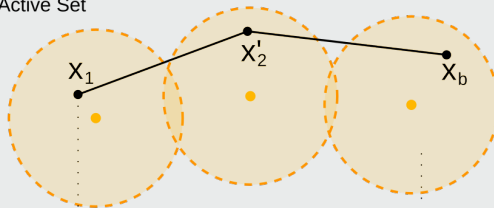
Inactive Set



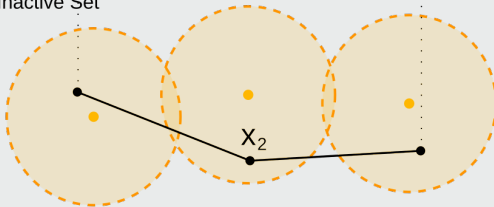
Sparse Stable Trees (SST)

Subtree pruning

Active Set



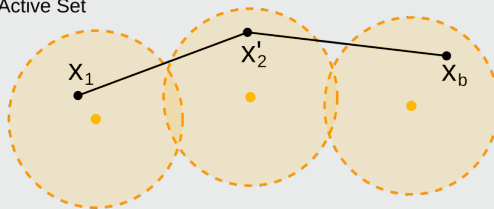
Inactive Set



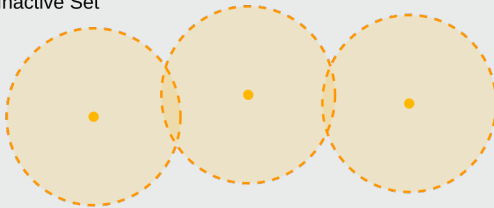
Sparse Stable Trees (SST)

Subtree pruning

Active Set



Inactive Set



Sparse Stable Trees (SST)

Properties

Sparse Stable Trees (SST*) is asymptotically near-optimal (AnO)

Asymptotically near-optimal

Planner finds a solution with cost at most $(1 + \epsilon)c^*$

Y Li, Z Littlefield, KE Bekris, "Asymptotically Optimal Sampling-based Kinodynamic Planning", 2016

Sparse Stable Trees (SST)

Advantages

- Selection always picks a locally optimal node
- Memory footprint is minimized

Sparse Stable Trees (SST)

Advantages

- Selection always picks a locally optimal node
- Memory footprint is minimized

Drawbacks

- Need to choose a good δ_S for the witness radii
- AnO, not AO

Optimal Planning for Kinodynamic Systems

AO-x

- AO-x is a meta algorithm
- Input is any feasible kinodynamic planner
- Idea: Convert the bounded-suboptimal version of optimal kinodynamic planning into a feasible kinodynamic problem
- Then iterate: Solve bounded-suboptimal version, compute best cost found, setup new bounded-suboptimal version with this cost, etc

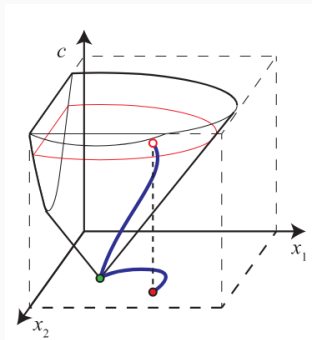
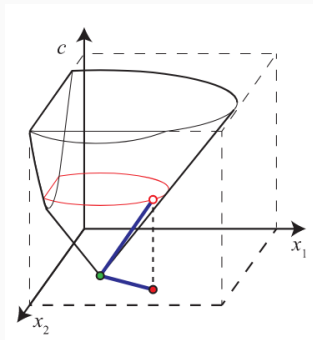
State-Cost Space

- Augment configuration space $\mathcal{Q}$ with a real cost dimension:

$$\mathcal{Q}' = \mathcal{Q} \times \mathbb{R}_0^+$$

- Augment dynamics $\mathbf{f}$, where $\mathbf{q}' = (\mathbf{q}, c)$:

$$\mathbf{f}'(\mathbf{q}', \mathbf{u}) = \left(\mathbf{f}(\mathbf{q}, \mathbf{u}), \Delta c \right)$$



Algorithm 2: Asymptotically-optimal($\mathcal{P}, \mathcal{A}, n$).

```
1: Run  $\mathcal{A}(\mathcal{P}_\infty)$  to obtain a first path  $y_0$ . If no solution  
   exists, report ' $\mathcal{P}$  has no solution.'  
2: Let  $c_0 = C(y_0)$ .  
3: for  $i = 1, 2, \dots, n$  do  
4:   Run  $\mathcal{A}(\mathcal{P}_{c_{i-1}})$  to obtain a new solution  $y_i$ .  
5:   Let  $c_i = C(y_i)$ .  
6: return  $y_n$ 
```

- $\mathcal{A}$ is typically (kinodynamic) RRT or EST
- $\mathcal{P}_{\bar{c}}$ is a problem instance with cost bound $\bar{c}$

AO-x: Proof of asymptotic optimality (AO)

Assumptions

1. $\mathcal{A}$ terminates in finite time, if solution within given bound $\bar{c}$ exists
2. $\mathcal{A}$ reduces cost by a nonnegligible amount.

$$E[c(y_i)|\bar{c}] - c^* \leq (1 - \omega)(\bar{c} - c^*) \quad \text{for } \omega > 0,$$

where c^* is the optimal cost and $\bar{c}$ the cost limit

AO-x: Proof of asymptotic optimality (AO)

Proof Goal

Let $S_0, \dots, S_n$ be random variables for $c(y_i) - c^*$. Then for any $\epsilon > 0$ we have:

$$\lim_{n \rightarrow \infty} P(S_n \geq \epsilon) = 0$$

Proof helpers:

- Markov inequality: $P(S_n \geq \epsilon) \leq E[S_n]/\epsilon$
- Assumption 2 (cost reduction by nonnegligible amount): $E[S_n | s_{n-1}] \leq (1 - \omega)s_{n-1}$

AO-x: Proof of asymptotic optimality (AO)

Use $E[S_n|s_{n-1}] \leq (1 - \omega)s_{n-1}$:

$$\begin{aligned} E[S_n] &= \int E[S_n|s_{n-1}]P(s_{n-1})ds_{n-1} \\ &\leq \int (1 - \omega)s_{n-1}P(s_{n-1})ds_{n-1} \\ &= (1 - \omega) \int s_{n-1}P(s_{n-1})ds_{n-1} \\ &= (1 - \omega)E[S_{n-1}] \\ &= (1 - \omega)^n E[S_0] \end{aligned}$$

AO-x: Proof of asymptotic optimality (AO)

Use $E[S_n|s_{n-1}] \leq (1 - \omega)s_{n-1}$:

$$\begin{aligned} E[S_n] &= \int E[S_n|s_{n-1}]P(s_{n-1})ds_{n-1} \\ &\leq \int (1 - \omega)s_{n-1}P(s_{n-1})ds_{n-1} \\ &= (1 - \omega) \int s_{n-1}P(s_{n-1})ds_{n-1} \\ &= (1 - \omega)E[S_{n-1}] \\ &= (1 - \omega)^n E[S_0] \end{aligned}$$

Use Markov inequality:

$$\begin{aligned} P(S_n \geq \epsilon) &\leq E[S_n]/\epsilon \\ P(S_n \geq \epsilon) &\leq (1 - \omega)^n E[S_0]/\epsilon \end{aligned}$$

AO-x: Proof of asymptotic optimality (AO)

Use $E[S_n|s_{n-1}] \leq (1 - \omega)s_{n-1}$:

$$\begin{aligned} E[S_n] &= \int E[S_n|s_{n-1}]P(s_{n-1})ds_{n-1} \\ &\leq \int (1 - \omega)s_{n-1}P(s_{n-1})ds_{n-1} \\ &= (1 - \omega) \int s_{n-1}P(s_{n-1})ds_{n-1} \\ &= (1 - \omega)E[S_{n-1}] \\ &= (1 - \omega)^n E[S_0] \end{aligned}$$

Use Markov inequality:

$$\begin{aligned} P(S_n \geq \epsilon) &\leq E[S_n]/\epsilon \\ P(S_n \geq \epsilon) &\leq (1 - \omega)^n E[S_0]/\epsilon \end{aligned}$$

Take the limit:

$$\begin{aligned} \lim_{n \rightarrow \infty} P(S_n \geq \epsilon) &\leq (1 - \omega)^n E[S_0]/\epsilon \\ &= 0 \end{aligned}$$

Advantages

- Planner agnostic
- Enhances theoretical properties (AO)

Advantages

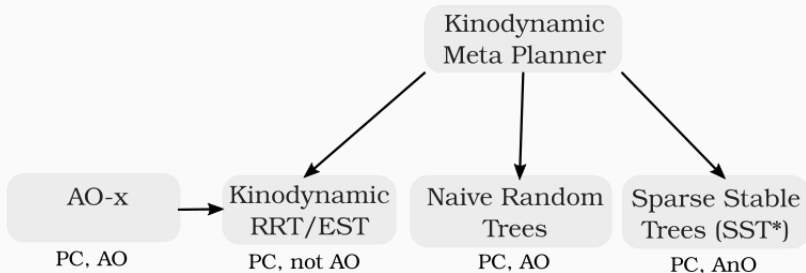
- Planner agnostic
- Enhances theoretical properties (AO)

Drawbacks

- By default, no re-use of data between iterations
- Unknown convergence rate (rather poor empirically)

Conclusion

- Kinodynamic planners: **kinodynamic RRT/EST**, **SST(*)**, **AO-x**



- Geometric planners: RRT, RRT*, BIT*

Next Time

- More planners (EST, RRT-Connect, PRM*, FMT*) and theoretical insights

1. Yanbo Li, Zakary Littlefield, and Kostas E. Bekris. **“Asymptotically Optimal Sampling-Based Kinodynamic Planning”**. In: *I. J. Robotics Res.* 35.5 (2016), pp. 528–564. DOI: 10.1177/0278364915614386
2. Kris Hauser and Yilun Zhou. **“Asymptotically Optimal Planning by Feasible Kinodynamic Planning in a State-Cost Space”**. In: *IEEE Trans. Robotics* 32.6 (2016), pp. 1431–1443. DOI: 10.1109/TR0.2016.2602363

- [1] Kiril Solovey, Lucas Janson, Edward Schmerling, Emilio Frazzoli, and Marco Pavone. **“Revisiting the Asymptotic Optimality of RRT*”**. In: *IEEE International Conference on Robotics and Automation (ICRA)*. May 2020, pp. 2189–2195. DOI: 10.1109/ICRA40945.2020.9196553.
- [2] Sertac Karaman and Emilio Frazzoli. **“Sampling-Based Algorithms for Optimal Motion Planning”**. In: *International Journal of Robotics Research (IJRR)* 30.7 (2011), pp. 846–894. DOI: 10.1177/0278364911406761.
- [3] Yanbo Li, Zakary Littlefield, and Kostas E. Bekris. **“Asymptotically Optimal Sampling-Based Kinodynamic Planning”**. In: *I. J. Robotics Res.* 35.5 (2016), pp. 528–564. DOI: 10.1177/0278364915614386.

- [4] Kris Hauser and Yilun Zhou. **“Asymptotically Optimal Planning by Feasible Kinodynamic Planning in a State-Cost Space”**. In: *IEEE Trans. Robotics* 32.6 (2016), pp. 1431–1443. DOI: 10.1109/TR0.2016.2602363.